

Working Mobile App Shell

Auth, Profile Management & Medical History Uploads — Backend Contract

PawGuard Platform | pawguard-backend (FastAPI + PostgreSQL + Redis + ARQ) | Prepared for Team Review

1. Scope of This Document

This backend (pawguard-backend) is a FastAPI REST API; no Flutter/React Native project files were included in the delivered package. This document specifies the exact backend contract the mobile shell should be built against so a Flutter or React Native team can wire up Auth, Profile, and Medical History Upload screens directly.

2. Auth — Endpoints (prefix /auth)

Endpoint	Purpose
POST /auth/register	Create account
POST /auth/login	RS256 JWT access token (15-min) + opaque refresh token
POST /auth/refresh	Exchange refresh token for new access token
POST /auth/logout, /logout-all	Revoke current / all sessions
GET/PUT /auth/me	Read / update the signed-in profile
GET /auth/sessions, DELETE /auth/sessions/{id}	List / revoke active device sessions
POST /auth/mfa/enroll + verify	TOTP-based multi-factor auth
GET/POST/DELETE /auth/oauth/accounts	Social login linking

3. Profile Management

Once authenticated, the app screen for “My Pets” maps directly to the companion_pet endpoints: create/list/get/update/delete a pet (POST/GET/PATCH/DELETE /companion-pets), each scoped to the signed-in owner via companion_pet:* permissions.

4. Medical History Uploads (Direct-to-S3 Pattern)

- App calls POST /companion-pets/{pet_id}/medical-files/upload-url — backend returns a presigned S3 URL.
- App uploads the file bytes straight to S3 using that URL (never through the API server — keeps large files off the app backend).
- App calls PUT /companion-pets/{pet_id}/medical-files/{file_id}/confirm — backend verifies the object exists in S3.
- App can then attach the file to a medical record via POST /companion-pets/{pet_id}/medical-records, or list uploaded files with GET /companion-pets/{pet_id}/medical-files.

5. Recommended Shell Structure

- Auth stack: Splash → Login/Register → MFA (if enrolled) → Home.
- Store the JWT access token in memory + refresh token in secure/encrypted storage (Keychain / EncryptedSharedPreferences).
- Home tabs: My Pets, Lost & Found, Vet Directory, Reminders — each maps 1:1 to a module documented elsewhere in this set.
- A background refresh task should call /auth/refresh before the 15-minute access token expires to keep the session seamless.

Basis: auth/router.py, companion_pet/router.py. No Flutter/React Native source was present in the uploaded package — confirm with the mobile team before treating this as an as-built spec.